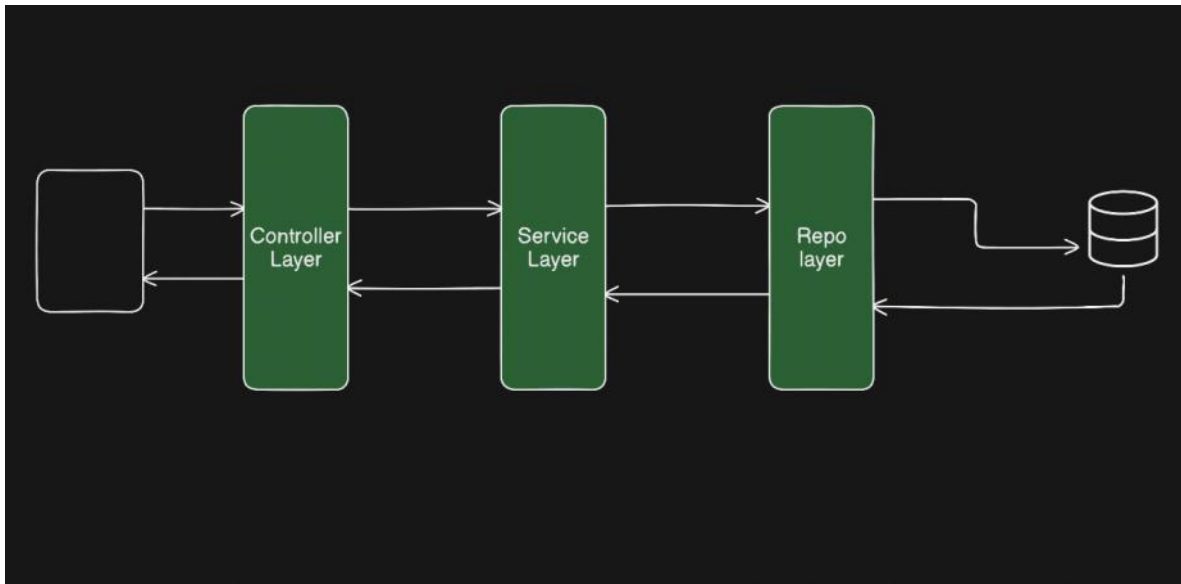


Spring Boot + JPA

05 September 2026 09:13



```
Connection connection = DriverManager.getConnection(URL, USER, PASSWORD)
PreparedStatement preparedStatement = connection.prepareStatement(sqlQuery)
preparedStatement.setString(1,"Rahul")
```

The code above will work, but in real project it become repetitive.

Problem with only JDBC:

1. We write SQL again n again
2. We manually convert java object data into table columns
3. We manually read ResultSet and create java object
4. Changing database table structure become painful.
5. Relationship like one student has many courses become difficult.

What is ORM ?

ORM means Object relational mapping

Object means java class object

Relational mean database table

Mapping means connecting java class fields with database table columns.

Java Class	Database table
Student	stdudents
Id	Id
Name	Name

City	city
------	------

Why we use ORM ?

We use ORM because developer want to work more with java object and less with database

When we use ORM ?

In Backend application where data must be **saved, update, delete and fetched** from database.

What is JPA ?

JPA means java persistence API.

JPA is not an implementation. JPA is specification, which mean it give rules and annotations:

JPA says:

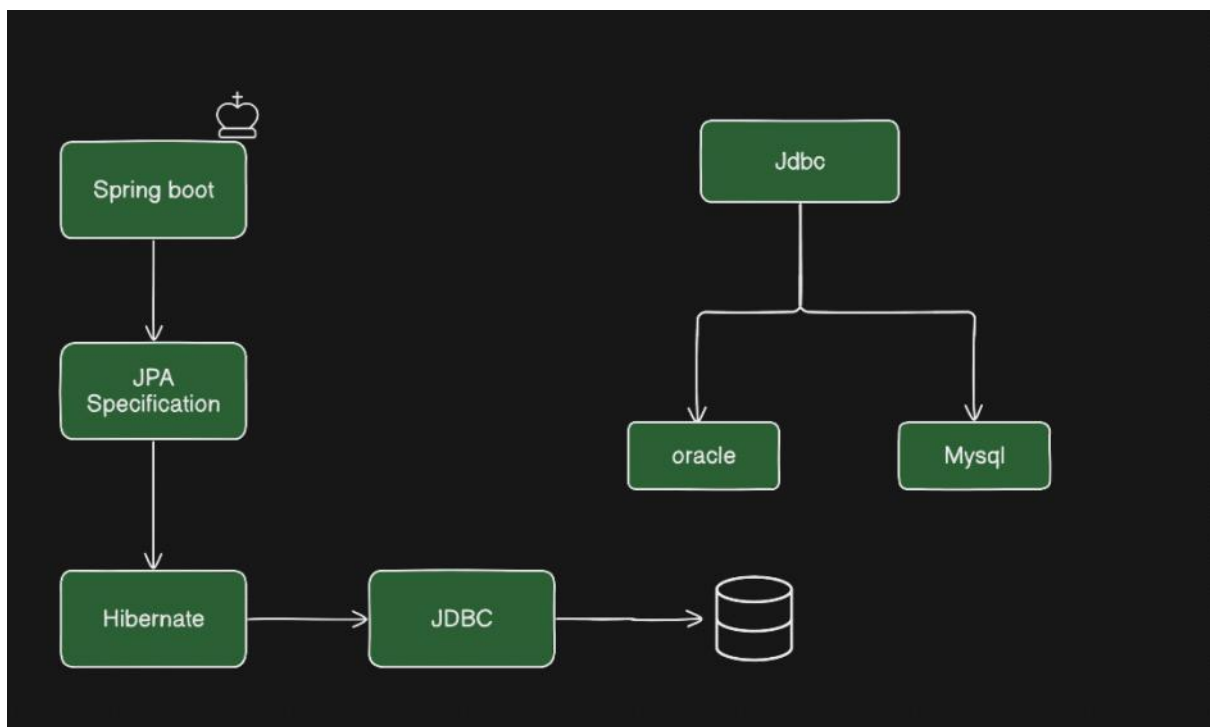
1. How to mark a class as database entity
2. How to mark primary key
3. How to create persistence operation
4. How transaction should work

But JPA alone does not run. We need implementation.

Popular JPA implementation = Hibernate

In most Spring project:

Spring Data JPA ---> JPA Specification ---> Hibernate ---> JDBC ---> Database



Important point :

JPA is like an interface
Hibernate is like the class that implement it
Spring data jpa make it even easier by giving repository interface

Interview Line :

JPA give the rules, Hibernate follow the rules, Spring data jpa reduce boilerplate code.

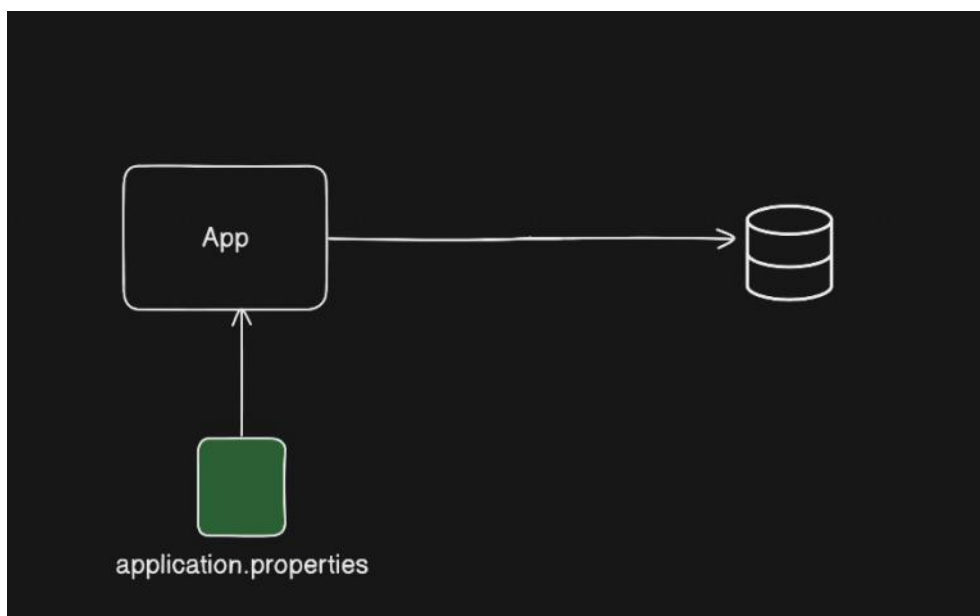
Required Dependency For JPA:

```
<dependencies>
<dependency>
  <groupId>org.springframework.data</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
</dependencies>
```

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.0.31</version>
  <scope>compile</scope>
</dependency>
```

Why we use spring-boot-starter-data-jpa ?

It brings Spring Data JPA, hibernate, transaction support and required JPA libraries.



What is entity ?

An entity is a java class whose object can be stored in database.

Why @Entity is used ?

It tells JPA this class should be mapped with a database table

Why @Id is used ?

Every entity must have a primary key .
JPA uses this primary key to identify one unique row.

```
@GeneratedValue(strategy = GenerationType.IDENTITY)  
private Long id;
```

Why ?

It tells JPA to generate id automatically

Common Strategies

Strategy	Meaning	Where used
IDENTITY	Database auto increment	MYSQL, PostgreSQL, Oracle
SEQUENCE	Used database sequence	Oracle, PostgreSQL
AUTO	Provide decides	Simple project
TABLE	Uses separate table for id	Rarely used

@Table

By default table name is same as class name if we want custom name then we will use @Table

@Column

To customize column name and constraints

Entity class rules

An entity class should:

1. Have @Entity
2. Have @Id
3. Have no-argument constructor
4. Not be final
5. Have fields with getter and setter

Why No-Argument constructor needed

Hibernate create entity object internally using reflection. If no default constructor is available, hibernate can face object creation problem.

Repository

1. CrudRepository
2. JpaRepository

Why repository used

Repository talk to database without writing implementation class, Spring data jpa give method.

1. Save()
2. findById()
3. findAll()
4. deleteById()
5. Count()

Repository is a marker interface. It does not provide method directly

CrudRepository

Use it when you requirement is only basic **create, read, update and delete**. Or in small services where pagination, sorting, batch flush and jpa-specific method are not required.

PagingAndShortingRepository

Use it when we need records page by page or sorted.

Example:

1. Product list page
2. Employee list page
3. Order history

JpaRepository

```
@Repository
public interface StudentRespository extends JpaRepository<Student,Long> {
}
```

JpaRepository is most commonly used in spring boot jpa projects.
It include CRUD, pagination, sorting and JPA-Specific operations.

Derived Query Method

Spring Data JPA can create query from method name.

Select * from student where city="Agra" and Name="Ashwani";

```

List<Student> findByCity(String city);

List<Student> findByName(String city);

List<Student> findByCityOrName(String city, String name);

List<Student> findByAgeGreaterThan(int age);

List<Student> findByNameContaining(String keyword);

List<Student> findByCityOrderByNameAsc(String city);

Boolean existsByEmail(String email)

long countByCity(String city)

Booking --> user --> id
Filter by user.id then sort by bookedAt

FindByUserIdOrderByBookedAtDesc(Long id)

```

@Query JPQL (Java Persistence query language)

JPQL works with entity name and field name, not table name and column name.

Why @Query is used ?

When derived method name become too long or query is custom.

Where to use ?

Use JPQL when query is related to entity objects.

```

@Query(" select s from student s where s.age >= :age ")
List<Student> findAdultsStudent(int age)

```

Native Query

Native query means actual SQL

```

@Query(value = "select * from student where city = :city", nativeQuery = true)
List<Student> findByCityNative(String city);

```

Updating data with @Modifying

Why ?

Spring Data JPA assume @Query is select query for update/delete query we use @Modifying

Why @Transaction Is needed

Update and delete must happen inside transaction

Transaction:

Transaction means all operations should complete successfully, otherwise all should rollback.

Real-life example

Money Transfer :

1. Deduct 500 from rahul account
2. Add 500 to aman account

If deduction succeeds but adding fail, money is lost

@Transactional

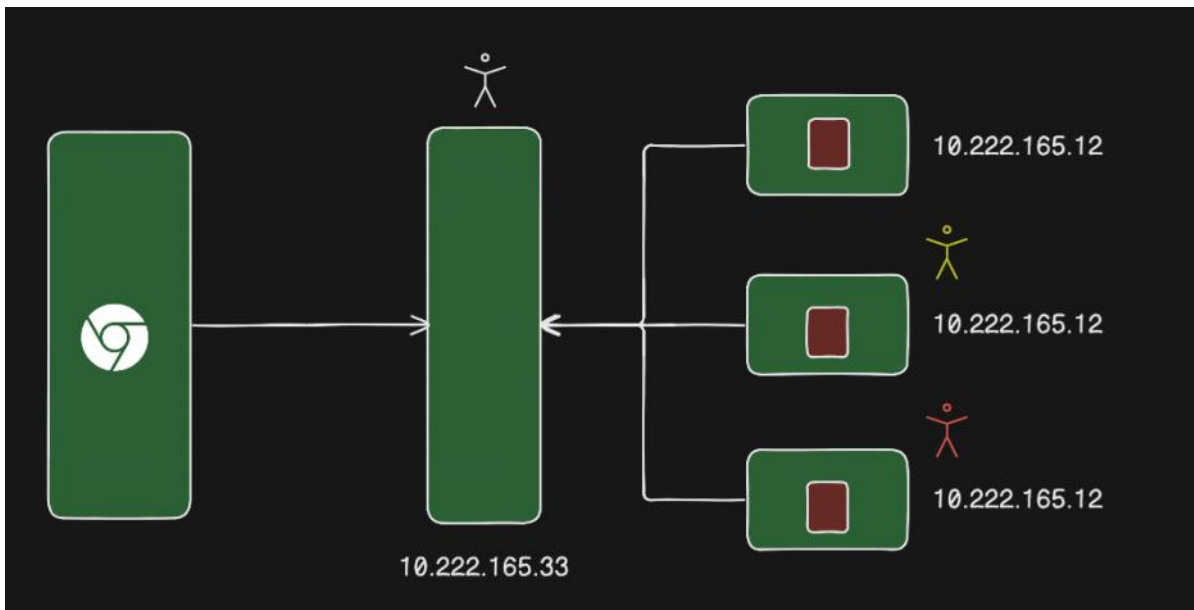
How @Transactional work internally

Most student says : @Transactional means rollback if error comes.

Example:

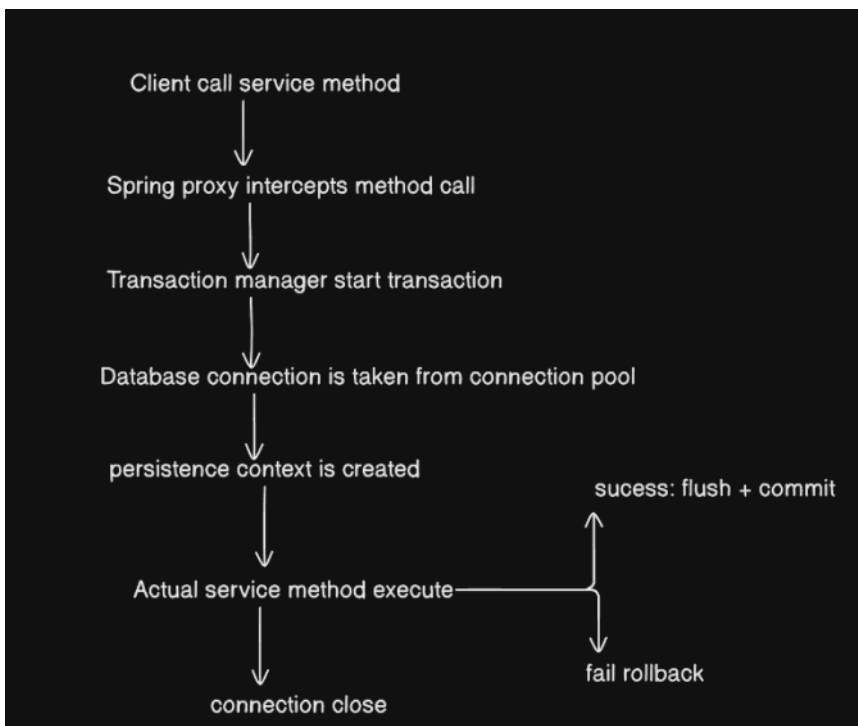
```
@Transactional
public void placeOrder(Request request)
{
    orderRepo.save(order); ---> done
    paymentRepo.save(payment); ----> done
    inventoryRepo.updateStock(order); ---> fail
}
```

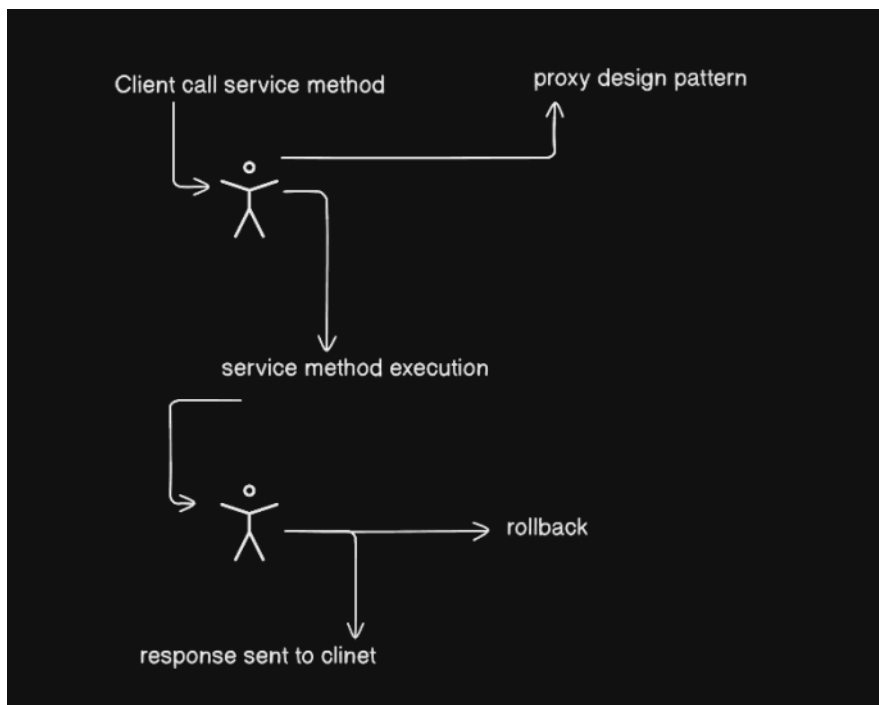
Problem: data become inconsistent



Internal flow of @Transactional

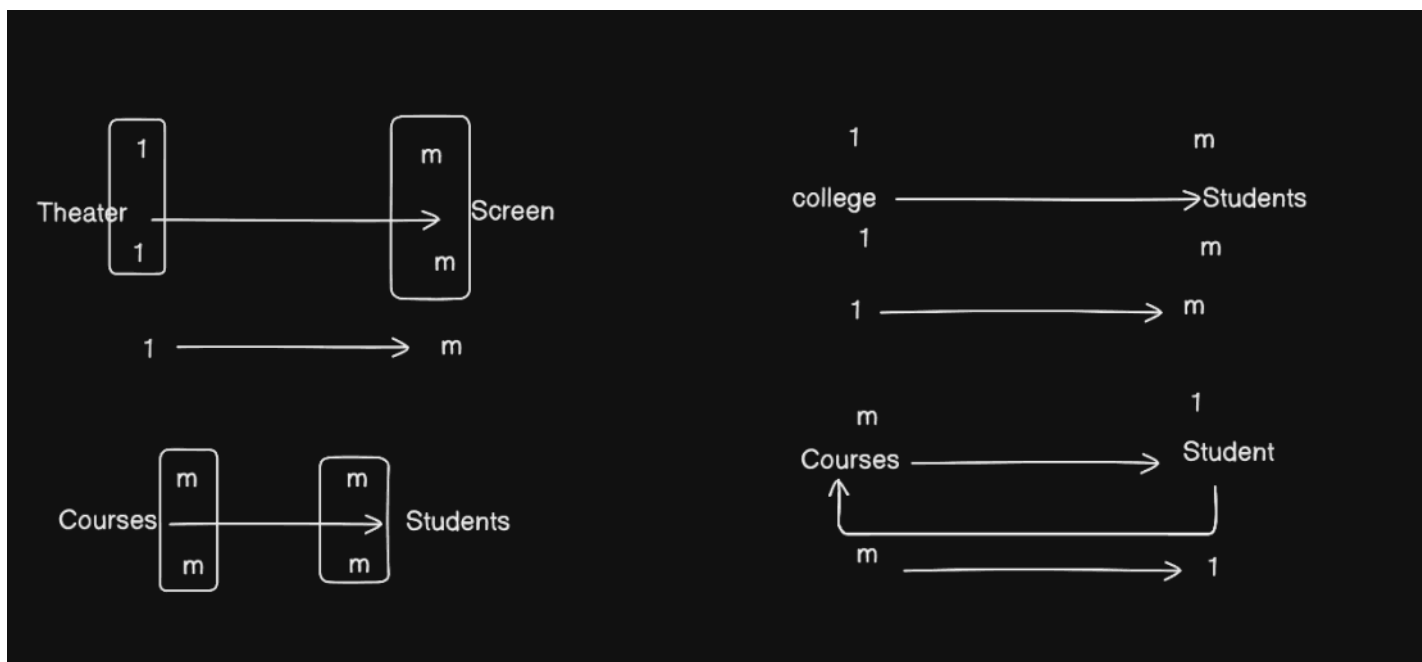
Client call service method





Relationship mappings

In real projects, table are connected



1. @OneToMany
2. @ManyToOne
3. @OneToOne
4. @ManyToMany